FAKE NEWS DETECTION USING TF-IDF

Project Documentation | Natural Language Processing and Machine Learning

1. Abstract

This project builds a text-classification pipeline to classify news articles as FAKE or REAL using their titles and article bodies. After basic preprocessing, article text is converted into numerical features with Term Frequency–Inverse Document Frequency (TF-IDF). Multiple machine-learning classifiers are trained and evaluated using a common stratified train-test split. In the notebook's final comparison, Linear Support Vector Machine (Linear SVM) performs best, achieving 93.26% accuracy and a 93.31% F1-score for the FAKE class.

2. Introduction

The large volume of online news makes it difficult to manually examine every article. Text classification can learn language patterns associated with labels in a training dataset and predict whether a new article resembles the FAKE or REAL examples. This project studies TF-IDF feature extraction and compares several machine-learning models. A model prediction is only an automated classification based on learned patterns; it does not independently prove whether the claims in an article are true.

3. Objectives

- Load and prepare a labelled news dataset.
- Combine each article's title and body into a single text field.
- Convert text into numerical features using TF-IDF.
- Train and compare multiple classification algorithms.
- Evaluate the models using accuracy, precision, recall, and F1-score.
- Identify the best-performing classifier among the tested models.

4. Dataset Description

The notebook reads news.csv, which initially contains 6,335 rows and four columns: Unnamed: 0, title, text, and label. The label column contains REAL and FAKE classes. The initial label counts are 3,171 REAL and 3,164 FAKE. After removing rows with missing labels, filling missing title/text fields, combining title and body, and dropping empty and duplicate content, 6,306 records remain.

Dataset detail	Value
Initial dataset size	6,335 rows × 4 columns
Records after preprocessing	6,306
Training records	5,044 (80%)
Testing records	1,262 (20%)
Test-set class counts	631 FAKE and 631 REAL
Split configuration	random_state=42, stratify=y

5. Data Preprocessing

- Remove records with missing labels.
- Replace missing titles and article text with empty strings.
- Concatenate title and text into a single content field.
- Remove records whose combined content is empty.
- Remove duplicate articles based on the content field.
- Normalize labels by trimming whitespace and converting them to uppercase.

The final target labels used by the models are FAKE and REAL. The feature input X contains combined article text, while y contains the corresponding labels.

6. Methodology

6.1 TF-IDF Feature Extraction

TF-IDF stands for Term Frequency–Inverse Document Frequency. It represents text numerically by assigning weights to words or word sequences. Term frequency reflects how often a term appears in a document, while inverse document frequency reduces the influence of terms that occur in many documents. The resulting weights help represent terms that are relatively informative for an article.

The notebook uses two TF-IDF configurations:

Initial Logistic Regression experiment: max_features=50,000, stop_words='english', ngram_range=(1, 2), min_df=2, sublinear_tf=True.

Model comparison experiment: max_features=30,000, stop_words='english', ngram_range=(1, 2), min_df=2, sublinear_tf=True.

Unigrams represent single words and bigrams represent adjacent two-word sequences. English stop words are excluded, terms appearing in fewer than two documents are filtered out, and sublinear term-frequency scaling is enabled. The vectorizer is fitted on the training data and then used to transform the test data.

6.2 Train-Test Split

The dataset is split into 80% training data and 20% testing data. The random_state value is set to 42 for reproducibility, and stratify=y helps preserve the class distribution in both subsets.

6.3 Machine-Learning Models

The notebook evaluates Logistic Regression in an initial TF-IDF experiment, then compares Multinomial Naive Bayes, Bernoulli Naive Bayes, Linear SVM, Random Forest, Decision Tree, and K-Nearest Neighbors (KNN) using the 30,000-feature TF-IDF representation. It also includes a separate Linear Regression experiment after mapping REAL to 1 and FAKE to 0; this is treated as a supplementary regression experiment rather than one of the main classification models.

7. Evaluation Metrics

Accuracy: The percentage of all test articles classified correctly.

Precision: Among articles predicted as a selected class, the proportion that truly belongs to that class.

Recall: Among all articles belonging to a selected class, the proportion correctly identified.

F1-score: The harmonic mean of precision and recall.

MAE: Mean absolute error for the supplementary Linear Regression experiment.

RMSE: Root mean squared error for the supplementary Linear Regression experiment.

R^2 score: A regression measure comparing model fit with a mean-target baseline.

In the final comparison table, precision, recall, and F1-score are calculated with FAKE as the positive class. The detailed classification reports also provide separate scores for FAKE and REAL.

8. Results and Model Comparison

Final comparison results reported in the supplied notebook:

Model	Accuracy	Precision*	Recall*	F1-score*
Linear SVM + TF-IDF	93.26%	92.66%	93.98%	93.31%
Multinomial NB + TF-IDF	90.17%	90.95%	89.22%	90.08%
KNN + TF-IDF	90.02%	94.07%	85.42%	89.53%
Random Forest + TF-IDF	88.91%	87.25%	91.13%	89.15%
Bernoulli NB + TF-IDF	83.44%	77.69%	93.82%	85.00%
Decision Tree + TF-IDF	80.27%	79.75%	81.14%	80.44%

Precision, recall, and F1-score use FAKE as the positive class.

Linear SVM gives the best result in the final comparison, with 93.26% accuracy and 93.31% F1-score for FAKE. Multinomial Naive Bayes and KNN also perform strongly, with accuracy near 90%. Bernoulli Naive Bayes has high FAKE recall (93.82%) but lower FAKE precision (77.69%), which means it identifies many FAKE articles but also labels more REAL articles as FAKE compared with models having higher precision.

9. Logistic Regression and Linear Regression Experiments

9.1 Logistic Regression

The first experiment uses TF-IDF with up to 50,000 features and LogisticRegression(max_iter=1000). It reports 92.63% accuracy. The classification report shows approximately 0.93 F1-score for both classes. This result comes from a different feature configuration than the later 30,000-feature model comparison, so it should be reported separately rather than directly mixed into the final comparison table.

9.2 Linear Regression

The notebook maps REAL to 1 and FAKE to 0 and fits LinearRegression to TF-IDF features. The reported metrics are MAE = 0.2093, RMSE = 0.2721, and R^2 = 0.7038. These are regression metrics and are not directly comparable with classification accuracy or F1-score. Classification algorithms are more suitable for the main REAL/FAKE prediction task.

10. Tools and Libraries

- Python
- Pandas and NumPy for data handling
- scikit-learn for TF-IDF, train-test splitting, machine-learning models, and evaluation
- Matplotlib and Seaborn for visualizations
- Joblib imported for model persistence
- Jupyter Notebook for implementation and experimentation

11. Project Workflow

1. Load the news.csv dataset.
2. Inspect the dataset and class distribution.
3. Clean missing values and duplicate article content.
4. Combine title and body into one text field.
5. Split the dataset into training and testing sets.
6. Fit TF-IDF on training text and transform training and test text.
7. Train classification models and generate predictions.
8. Calculate evaluation metrics and compare model performance.
9. Save the comparison table to tfid_comparison.csv.

12. Limitations

- The model learns patterns in the provided dataset and may not generalize to newer articles or other sources.
- A model label is not proof that an article's claims are true; fact-checking requires reliable external evidence.
- TF-IDF primarily captures word and phrase statistics and does not fully understand context, irony, or factual accuracy.
- The reported results use one train-test split; cross-validation and testing on an independent dataset could strengthen evaluation.
- The notebook does not establish the dataset's provenance or provide an external fact-checking validation set.

13. Conclusion

This project demonstrates a news text-classification pipeline using TF-IDF and multiple machine-learning algorithms. After preprocessing 6,306 articles and evaluating models on a 1,262-article test set, Linear SVM achieves the strongest result in the final comparison: 93.26% accuracy and a 93.31% F1-score for the FAKE class. The initial Logistic Regression experiment

reports 92.63% accuracy using a separate TF-IDF configuration. Further cross-validation and independent evaluation are recommended before using the system in real-world settings.

Appendix: Main TF-IDF Configuration

TfidfVectorizer(max_features=30000, stop_words='english', ngram_range=(1, 2), min_df=2, sublinear_tf=True)

In [1]:
```
#import libraries
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

import matplotlib.pyplot as plt
import seaborn as sns
import joblib
```

In [2]:
```
# load dataset
df = pd.read_csv("news.csv")

print("Dataset shape:", df.shape)
print(df.head())
print(df["label"].value_counts())
```

```
Dataset shape: (6335, 4)
   Unnamed: 0                                              title  \
0        8476                       You Can Smell Hillary’s Fear
1       10294  Watch The Exact Moment Paul Ryan Committed Pol...
2        3608        Kerry to go to Paris in gesture of sympathy
3       10142  Bernie supporters on Twitter erupt in anger ag...
4         875   The Battle of New York: Why This Primary Matters

                                                text label
0  Daniel Greenfield, a Shillman Journalism Fello...  FAKE
1  Google Pinterest Digg Linkedin Reddit Stumbleu...  FAKE
2  U.S. Secretary of State John F. Kerry said Mon...  REAL
3  — Kaydee King (@KaydeeKing) November 9, 2016 T...  FAKE
4  It's primary day in New York and front-runners...  REAL
label
REAL    3171
FAKE    3164
Name: count, dtype: int64
```

In [3]:
```
# data preprocessing
df = df.dropna(subset=["label"]).copy()

df["title"] = df["title"].fillna("")
df["text"] = df["text"].fillna("")

df["content"] = (
    df["title"].astype(str) + " " +
    df["text"].astype(str)
).str.strip()

df = df[df["content"] != ""]
df = df.drop_duplicates(subset=["content"])
```

```
X = df["content"]
y = df["label"].str.strip().str.upper()

print("Total articles:", len(df))
print("Labels:", y.unique())
print("Missing values:", X.isna().sum())
```

```
Total articles: 6306
Labels: <ArrowStringArray>
['FAKE', 'REAL']
Length: 2, dtype: str
Missing values: 0
```

In [4]:
```
#split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 5044
Testing samples: 1262
```

In [5]:
```
#build the TF-IDF model
tfidf_vectorizer = TfidfVectorizer(
    max_features=50000,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

print("Training feature shape:", X_train_tfidf.shape)
print("Testing feature shape:", X_test_tfidf.shape)
```

```
Training feature shape: (5044, 50000)
Testing feature shape: (1262, 50000)
```

In [6]:
```
#training
tfidf_model = LogisticRegression(max_iter=1000)

tfidf_model.fit(X_train_tfidf, y_train)

tfidf_predictions = tfidf_model.predict(X_test_tfidf)

print("TF-IDF Accuracy:",
      f"{accuracy_score(y_test, tfidf_predictions):.2%}")

print(classification_report(y_test, tfidf_predictions))
```

```
TF-IDF Accuracy: 92.63%
              precision    recall  f1-score   support

        FAKE       0.92      0.94      0.93       631
        REAL       0.94      0.91      0.93       631

    accuracy                           0.93      1262
   macro avg       0.93      0.93      0.93      1262
weighted avg       0.93      0.93      0.93      1262
```

In [7]:

```python
import pandas as pd
import numpy as np
import time
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer, HashingVecto

from sklearn.naive_bayes import MultinomialNB, BernoulliNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix,
    mean_absolute_error, mean_squared_error, r2_score
)

df = pd.read_csv("news.csv")

df = df.dropna(subset=["label"]).copy()
df["title"] = df["title"].fillna("")
df["text"] = df["text"].fillna("")
df["label"] = df["label"].str.strip().str.upper()

df["content"] = (
    df["title"].astype(str) + " " + df["text"].astype(str)
).str.strip()

df = df[df["content"] != ""]
df = df[df["label"].isin(["REAL", "FAKE"])]
df = df.drop_duplicates(subset=["content"])

X = df["content"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Total records:", len(df))
print("Training records:", len(X_train))
print("Testing records:", len(X_test))
```

```
Total records: 6306
Training records: 5044
Testing records: 1262
```

In [8]:
```python
# Create TF-IDF features
tfidf_vectorizer = TfidfVectorizer(
    max_features=30000,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

print("Training shape:", X_train_tfidf.shape)
print("Testing shape:", X_test_tfidf.shape)
```

```
Training shape: (5044, 30000)
Testing shape: (1262, 30000)
```

In [9]:
```python
# Multinomial Naive Bayes
mnb_tfidf = MultinomialNB()
mnb_tfidf.fit(X_train_tfidf, y_train)

pred_mnb_tfidf = mnb_tfidf.predict(X_test_tfidf)

print("TF-IDF + Multinomial Naive Bayes")
print("Accuracy:", accuracy_score(y_test, pred_mnb_tfidf))
print(classification_report(y_test, pred_mnb_tfidf))
```

```
TF-IDF + Multinomial Naive Bayes
Accuracy: 0.901743264659271
              precision    recall  f1-score   support

        FAKE       0.91      0.89      0.90       631
        REAL       0.89      0.91      0.90       631

    accuracy                           0.90      1262
   macro avg       0.90      0.90      0.90      1262
weighted avg       0.90      0.90      0.90      1262
```

In [10]:
```python
# Bernoulli Naive Bayes
bnb_tfidf = BernoulliNB()
bnb_tfidf.fit(X_train_tfidf, y_train)

pred_bnb_tfidf = bnb_tfidf.predict(X_test_tfidf)

print("TF-IDF + Bernoulli Naive Bayes")
print("Accuracy:", accuracy_score(y_test, pred_bnb_tfidf))
print(classification_report(y_test, pred_bnb_tfidf))
```

```
TF-IDF + Bernoulli Naive Bayes
Accuracy: 0.8343898573692552
              precision    recall  f1-score   support

        FAKE       0.78      0.94      0.85       631
        REAL       0.92      0.73      0.82       631

    accuracy                           0.83      1262
   macro avg       0.85      0.83      0.83      1262
weighted avg       0.85      0.83      0.83      1262
```

In [11]:
```python
# Linear SVM
svm_tfidf = LinearSVC()
svm_tfidf.fit(X_train_tfidf, y_train)

pred_svm_tfidf = svm_tfidf.predict(X_test_tfidf)

print("TF-IDF + Linear SVM")
print("Accuracy:", accuracy_score(y_test, pred_svm_tfidf))
print(classification_report(y_test, pred_svm_tfidf))
```

```
TF-IDF + Linear SVM
Accuracy: 0.9326465927099842
              precision    recall  f1-score   support

        FAKE       0.93      0.94      0.93       631
        REAL       0.94      0.93      0.93       631

    accuracy                           0.93      1262
   macro avg       0.93      0.93      0.93      1262
weighted avg       0.93      0.93      0.93      1262
```

In [12]:
```python
# Random Forest
rf_tfidf = RandomForestClassifier(
    n_estimators=100,
    max_depth=30,
    random_state=42,
    n_jobs=-1
)

rf_tfidf.fit(X_train_tfidf, y_train)
pred_rf_tfidf = rf_tfidf.predict(X_test_tfidf)

print("TF-IDF + Random Forest")
print("Accuracy:", accuracy_score(y_test, pred_rf_tfidf))
print(classification_report(y_test, pred_rf_tfidf))
```

```
TF-IDF + Random Forest
Accuracy: 0.8890649762282092
              precision    recall  f1-score   support

        FAKE       0.87      0.91      0.89       631
        REAL       0.91      0.87      0.89       631

    accuracy                           0.89      1262
   macro avg       0.89      0.89      0.89      1262
weighted avg       0.89      0.89      0.89      1262
```

In [13]:
```
# Decision Tree
dt_tfidf = DecisionTreeClassifier(
    max_depth=30,
    random_state=42
)

dt_tfidf.fit(X_train_tfidf, y_train)
pred_dt_tfidf = dt_tfidf.predict(X_test_tfidf)

print("TF-IDF + Decision Tree")
print("Accuracy:", accuracy_score(y_test, pred_dt_tfidf))
print(classification_report(y_test, pred_dt_tfidf))
```

```
TF-IDF + Decision Tree
Accuracy: 0.8026941362916006
              precision    recall  f1-score   support

        FAKE       0.80      0.81      0.80       631
        REAL       0.81      0.79      0.80       631

    accuracy                           0.80      1262
   macro avg       0.80      0.80      0.80      1262
weighted avg       0.80      0.80      0.80      1262
```

In [14]:
```
# K-Nearest Neighbors (KNN)
knn_tfidf = KNeighborsClassifier(
    n_neighbors=5,
    algorithm="brute",
    metric="cosine"
)

knn_tfidf.fit(X_train_tfidf, y_train)
pred_knn_tfidf = knn_tfidf.predict(X_test_tfidf)

print("TF-IDF + KNN")
print("Accuracy:", accuracy_score(y_test, pred_knn_tfidf))
print(classification_report(y_test, pred_knn_tfidf))
```

```
TF-IDF + KNN
Accuracy: 0.9001584786053882
              precision    recall  f1-score   support

        FAKE       0.94      0.85      0.90       631
        REAL       0.87      0.95      0.90       631

    accuracy                           0.90      1262
   macro avg       0.90      0.90      0.90      1262
weighted avg       0.90      0.90      0.90      1262
```

In [15]:
```
# Linear Regression
y_train_num = (y_train == "REAL").astype(int)
y_test_num = (y_test == "REAL").astype(int)

lr_tfidf = LinearRegression()
lr_tfidf.fit(X_train_tfidf, y_train_num)

pred_lr_tfidf = lr_tfidf.predict(X_test_tfidf)
```

```
print("TF-IDF + Linear Regression")
print("MAE:", mean_absolute_error(y_test_num, pred_lr_tfidf))
print("RMSE:", np.sqrt(mean_squared_error(y_test_num, pred_lr_tfidf)))
print("R2 Score:", r2_score(y_test_num, pred_lr_tfidf))
```

```
TF-IDF + Linear Regression
MAE: 0.2092759066463458
RMSE: 0.2721007038558713
R2 Score: 0.7038448278445577
```

In [20]:

```
comparison_data = []

predictions = {
    "Multinomial NB - TF-IDF": pred_mnb_tfidf,

    "Bernoulli NB - TF-IDF": pred_bnb_tfidf,

    "Linear SVM - TF-IDF": pred_svm_tfidf,

    "Random Forest - TF-IDF": pred_rf_tfidf,

    "Decision Tree - TF-IDF": pred_dt_tfidf,

    "KNN - TF-IDF": pred_knn_tfidf,

}

for name, pred in predictions.items():
    comparison_data.append({
        "Model": name,
        "Accuracy (%)": round(accuracy_score(y_test, pred) * 100, 2),
        "Precision (%)": round(precision_score(
            y_test, pred, pos_label="FAKE", zero_division=0
        ) * 100, 2),
        "Recall (%)": round(recall_score(
            y_test, pred, pos_label="FAKE", zero_division=0
        ) * 100, 2),
        "F1 Score (%)": round(f1_score(
            y_test, pred, pos_label="FAKE", zero_division=0
        ) * 100, 2)
    })

comparison_df = pd.DataFrame(comparison_data)
comparison_df = comparison_df.sort_values(
    "F1 Score (%)", ascending=False
).reset_index(drop=True)

display(comparison_df)
comparison_df.to_csv("tfid_comparison.csv", index=False)
```

	Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
0	Linear SVM - TF-IDF	93.26	92.66	93.98	93.31
1	Multinomial NB - TF-IDF	90.17	90.95	89.22	90.08
2	KNN - TF-IDF	90.02	94.07	85.42	89.53
3	Random Forest - TF-IDF	88.91	87.25	91.13	89.15
4	Bernoulli NB - TF-IDF	83.44	77.69	93.82	85.00
5	Decision Tree - TF-IDF	80.27	79.75	81.14	80.44

In [ ]:

In [ ]: